

Custom Toast Overlay Documentation (Flutter)

Overview

This documentation explains a custom toast implementation in Flutter using `Overlay`, `AnimationController`, and transition animations. It allows you to display animated toast messages without relying on third-party packages.

Purpose

- Display temporary toast messages
 - Show smooth animations (fade + slide)
 - Avoid external dependencies
 - Provide full UI control
-

Full Source Code

```
import 'package:thedoctor0101/constants/common_imports.dart';

class _ToastOverlay extends StatefulWidget {
  final String message;
  final VoidCallback onDismissed;

  const _ToastOverlay({
    required this.message,
    required this.onDismissed,
  });

  @override
  State<_ToastOverlay> createState() => _ToastOverlayState();
}

class _ToastOverlayState extends State<_ToastOverlay>
    with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<double> _opacity;
  late Animation<Offset> _slide;

  @override
  void initState() {
    super.initState();
```

```

_controller = AnimationController(
  vsync: this,
  duration: const Duration(milliseconds: 300),
);

_opacity = Tween(begin: 0.0, end: 1.0).animate(_controller);

_slide = Tween(
  begin: const Offset(0, -0.3),
  end: Offset.zero,
).animate(_controller);

_controller.forward();

// Auto dismiss after 2 seconds
Future.delayed(const Duration(seconds: 2), () async {
  await _controller.reverse();
  widget.onDismissed();
});
}

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Positioned(
    top: 80,
    left: 20,
    right: 20,
    child: Material(
      color: Colors.transparent,
      child: FadeTransition(
        opacity: _opacity,
        child: SlideTransition(
          position: _slide,
          child: Center(
            child: Container(
              padding: const EdgeInsets.symmetric(
                horizontal: 16,
                vertical: 10,
              ),
              decoration: BoxDecoration(
                color: Colors.black.withValues(alpha: 0.85),

```

```

        borderRadius: BorderRadius.circular(10),
      ),
      child: Text(
        widget.message,
        style: const TextStyle(
          color: Colors.white,
          fontSize: 14,
        ),
      ),
    ),
  ),
),
),
),
);
}
}

void showCustomToast(BuildContext context, String message) {
  final overlay = Overlay.of(context, rootOverlay: true);

  late OverlayEntry overlayEntry;

  overlayEntry = OverlayEntry(
    builder: (context) => _ToastOverlay(
      message: message,
      onDismissed: () {
        overlayEntry.remove();
      },
    ),
  );

  overlay.insert(overlayEntry);
}

```

How It Works

1. Overlay System

- Uses `OverlayEntry` to render UI above everything
- No need to modify existing widget tree

2. Animation Controller

- Controls animation lifecycle

- Duration: 300ms

3. Animations Used

Fade Animation

```
_opacity = Tween(begin: 0.0, end: 1.0).animate(_controller);
```

- Smooth fade-in and fade-out

Slide Animation

```
_slide = Tween(  
  begin: const Offset(0, -0.3),  
  end: Offset.zero,  
) .animate(_controller);
```

- Toast slides from top to position

Lifecycle Flow

1. Toast is created using `showCustomToast()`
2. Overlay is inserted
3. Animation starts (`forward()`)
4. Toast stays visible for 2 seconds
5. Reverse animation starts
6. Overlay is removed

Usage Example

```
showCustomToast(context, "This is a custom toast!");
```

Customization Options

- Change position (`top` , `left` , `right`)
- Adjust duration (animation & delay)
- Modify colors and styles
- Add icons or actions inside toast

Best Practices

- Always remove overlay to prevent memory leaks
 - Keep toast messages short
 - Avoid stacking too many overlays
 - Use for non-critical messages only
-

Notes

- This approach gives full UI control
 - Works better than default SnackBar for custom designs
 - Fully reusable across the app
-

 This document is editable. You can extend it with UI screenshots, performance notes, or advanced features.